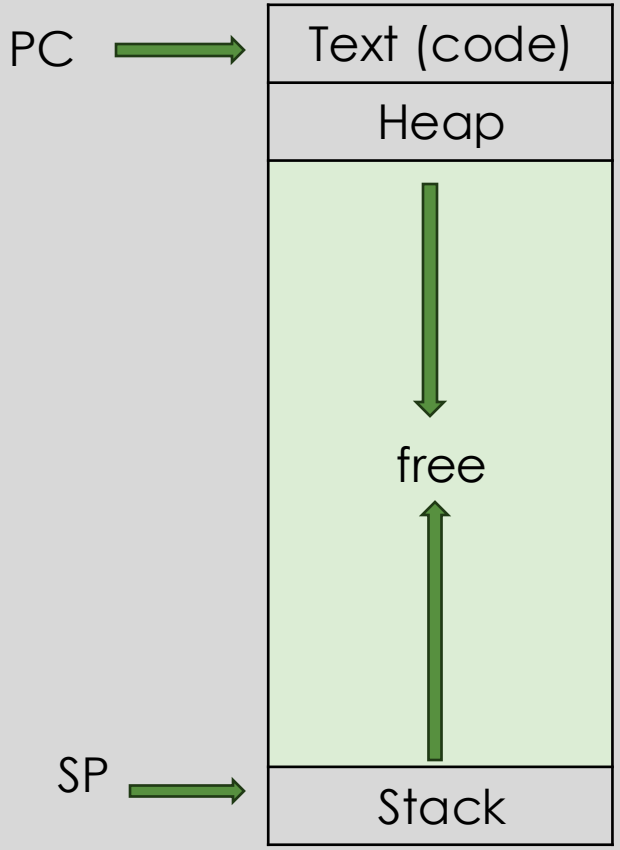

THREADS AND PARALLELISM

Single thread Process

- Single-threaded programs.
- Recap: Process Execution
  - The Program Counter (PC) points to the current instruction being executed.
  - The Stack Pointer (SP) points to the stack frame of the current function call.
- A program can also have multiple threads of execution.
- What is a thread?
- A thread is a unit of execution within a process.



Multi threaded process

- A **thread** is an independent unit of execution, similar to a lightweight copy of a process.
- **Threads share the same address space**, including:
  - Code segment
  - Heap (dynamically allocated memory)
- Each thread has its own:
  - **Program Counter (PC)** — so different threads can execute different parts of the program simultaneously.
  - **Stack** — allowing independent function calls and local variables.

Process vs Threads

- Parent P forks a child C
  - P and C do not share any memory
  - Need complicated IPC mechanisms to communicate
  - Extra copies of code, data in memory
- Parent P executes two threads T1 and T2
  - T1 and T2 share parts of the address space
  - Global variables can be used for communication
  - Smaller memory footprint
- Threads are like separate processes, except they share the same address space

Parallelism and Concurrency

- **Parallelism** allows a single process to efficiently use **multiple CPU cores** by running different threads or processes **simultaneously** on separate cores.
- It's important to distinguish between **concurrency** and **parallelism**:
  - **Concurrency** means multiple threads or processes make progress **at the same time**, even on a **single core**, by **interleaving** their execution.
  - **Parallelism** involves multiple threads or processes **actually executing at the same time** on **different CPU cores**.
- Even when parallelism isn't possible, concurrency is useful — for example, when one thread is **blocked on I/O**, other threads can continue running, keeping the CPU **efficiently utilized**.

Thread Scheduling and Management

- The **OS schedules threads** that are ready to run independently, just like it schedules processes.
- A thread's context (such as the **PC** and **registers**) is saved to and restored from a **Thread Control Block (TCB)**.
- Each **Process Control Block (PCB)** can be linked to one or more TCBs.
- Threads managed and scheduled directly by the **kernel** are called **kernel threads**.
  - For example, **Linux pthreads** are implemented as kernel threads.
- On the other hand, some libraries implement **user-level threads**:
  - The user program sees multiple threads, but the library maps many user threads onto fewer kernel threads.
  - Switching between user threads is **faster** because it avoids a full context switch.
  - However, user threads **cannot run in parallel**, since the kernel is unaware of them and only schedules the underlying kernel thread.

```c
#include <stdio.h>
#include <assert.h>
#include <pthread.h>
#include "common.h"
#include "common_threads.h"

void *mythread(void *arg) {
    printf("%s\n", (char *) arg);
    return NULL;
}

int
main(int argc, char *argv[]) {
    pthread_t p1, p2;
    int rc;
    printf("main: begin\n");
    Pthread_create(&p1, NULL, mythread, "A");
    Pthread_create(&p2, NULL, mythread, "B");
    // join waits for the threads to finish
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("main: end\n");
    return 0;
}
```

Creating threads using thread APi

```c
static volatile int counter = 0;

// mythread()
//
// Simply adds 1 to counter repeatedly, in a loop
// No, this is not how you would add 10,000,000 to
// a counter, but it shows the problem nicely.
//
void *mythread(void *arg) {
    printf("%s: begin\n", (char *) arg);
    int i;
    for (i = 0; i < 1e7; i++) {
        counter = counter + 1;
    }
    printf("%s: done\n", (char *) arg);
    return NULL;
}

// main()
//
// Just launches two threads (pthread_create)
// and then waits for them (pthread_join)
//
int main(int argc, char *argv[]) {
    pthread_t p1, p2;
    printf("main: begin (counter = %d)\n", counter);
    Pthread_create(&p1, NULL, mythread, "A");
    Pthread_create(&p2, NULL, mythread, "B");

    // join waits for the threads to finish
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("main: done with both (counter = %d)\n",
           counter);
    return 0;
}
```

Example: Threads with shared Data

- What do we expect? Two threads, each increments counter by 10^7, so 2X10^7

```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
```

- Sometimes, a lower value. Why?

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19345221)
```

Thread with shared data: what happens?

Digging deeper into shared data access

- The C statement `counter = counter + 1` may look simple, but it translates to **multiple machine-level instructions**.
- Here's what typically happens during execution:
  1. **Load** the value of `counter` from memory into a register.
  2. **Increment** the value in the register.
  3. **Store** the updated value back into the memory location of `counter`.

```
load counter → reg
reg = reg + 1
store reg → counter
```

- This multi-step process can lead to **race conditions** when multiple threads access and modify the same variable concurrently.

Motivation- Mutual Exclusion

Initially: A = 0;

T_0

A = A+1;

T_1

A=A+1;

T_0

```
ld R1, A
daddi R1, R1, #1
sd R1, A
```

T_1

```
ld R2, A
daddi R2, R1, #1
sd R2, A
```

Possible execution order

```
ld R1,A
ld R2, A
daddi R2, R1, #1
sd R2, A
daddi R1, R1, #1
sd R1, A
```

Race conditions & Mutual Exclusion

- When code runs incorrectly because of concurrency, it's called a **race condition**.
- This happens when **context switches** occur at the wrong time, breaking the **atomicity** of data updates.
- Race conditions occur during **concurrent access to shared data**, such as:
  - Threads sharing common data in the **user process memory**.
  - Processes running in **kernel mode** sharing **OS data structures**.
- To prevent such issues, we need to ensure **mutual exclusion** in certain parts of the code.
- That means **only one thread or process** should be allowed to run that part at a time.
- These parts of the program are called **critical sections** — they must run alone to work correctly.
  - Critical sections exist in **multi-threaded programs** and **OS code**.
- So, how do we protect critical sections?
  → **By using locks** (we'll learn more about this in the next topic).

Thank You